

Sesli Oda Senkronizasyon Raporu (Web + Flutter)

Amaç

Sesli oda sisteminin **web ve Flutter uygulamasında birebir aynı** çalışması. Tek backend, tek PostgreSQL veritabanı. Flutter tarafında ayrı iş mantığı yok — her şey aynı backend API'leri üzerinden yürüyor. Web tarafı bozulmadı; tüm değişiklikler **eklemeli (additive)** yapıldı, veritabanı şeması değişmedi (migration gerekmez).

Aynı tablolar kullanılıyor: `ChatRoom` (oda), `ChatPresence` (kim odada + koltuk), `ChatUserRole` (roller), `VoiceSession` (mikrofon/ses oturumu).

1. Kullanılan (mevcut) endpoint'ler

Bunlar zaten vardı ve hem web hem Flutter tarafından kullanılıyor. Hepsi çift kimlik doğrulamalıdır (web oturumu **veya** mobil JWT — `authenticateRequest`).

Endpoint	Metod	İşlev
/api/chat/rooms	GET	Oda listesi (aynı liste, sıralama, çevrimiçi sayısı, kapak görseli)
/api/chat/rooms/[roomId]/presence	GET	Odadaki aktif kullanıcılar + rolleri + koltuk
/api/chat/rooms/[roomId]/presence	POST	Odaya katılma / heartbeat / koltuk güncelleme
/api/chat/rooms/[roomId]/presence	DELETE	Odadan ayrılma
/api/chat/rooms/[roomId]/seats	PATCH	Koltuk değiştirme / atama
/api/chat/rooms/[roomId]/voice	GET/POST	Mikrofon (ses) oturumu aç/kapat, aktif ses listesi
/api/chat/rooms/[roomId]/stream	GET (SSE)	Gerçek zamanlı olay akışı
/api/chat/rooms/[roomId]/transfer-ownership	POST	Oda sahipliğini devretme
/api/trtc/token	POST	Flutter için TRTC UserSig üretimi
/api/trtc/usersig	POST	Web için TRTC UserSig üretimi
/api/admin/chat-rooms	GET/POST/PUT/DELETE	Yönetim: oda oluştur/güncelle/kapat/sil

2. Eklenen (yeni) endpoint'ler

Endpoint	Metod	İşlev
/api/chat/rooms/[roomId]/state	GET	Flutter için tek kaynaklı durum anlık görüntüsü. Tek istekte: oda bilgisi, katılımcılar (koltuk + mikrofons + rol + sıralama), 15'li koltuk haritası, çağırans kullanıcıns kendi durumu (me) ve TRTC bağlantı bilgisi (sdkAppId , trtcRoomId , numericUid).
/api/chat/rooms/[roomId]/seats	GET	Standart koltuk haritası — 15 koltuk (0-14). Her koltuk boş (null) ya da içindeki kullanıcı (id, isim, avatar, mikrofons durumu).

3. Düzenlenen endpoint'ler (eklemeli değişiklikler)

/api/chat/rooms/[roomId]/stream (SSE)

- Yeni gerçek zamanlı olayları type: "room_event" olarak yayınlar (aşağıdaki liste).
- presence olayına her kullanıcı için seatIndex , micOn ve image (avatar) alanları eklendi. Artık koltuk ve mikrofons durumu canlı akıyor.

/api/chat/rooms/[roomId]/presence (POST/DELETE)

- **Hayalet kullanıcı önleme:** Bir kullanıcı yeni bir odaya girdiğinde, önceki odalardaki varlığı (presence) ve mikrofons oturumları otomatik kapatılır. Böylece bir kullanıcı aynı anda iki odada görünmez (oda değiştirince eskisinden anında düşer).
- Katılma/ayrılmada user_joined / user_left olayları yayınlanır.
- Ayrılmada odadaki VoiceSession (mikrofons) da pasifleştirilir.

/api/chat/rooms/[roomId]/seats (PATCH)

- Koltuk değişiminden sonra seat_changed olayı yayınlanır (eski + yeni koltuk).

/api/chat/rooms/[roomId]/voice (POST)

- Mikrofons açılınca mic_changed (micOn:true) , kapanınca mic_changed (micOn:false) yayınlanır.
- Sayısal UID hesabı ortak lib/trtc-room.ts içine taşındı (web/Flutter'da birebir aynı).

/api/chat/rooms/[roomId]/transfer-ownership (POST)

- Sahiplik devrinden sonra owner_changed olayı yayınlanır.

/api/admin/chat-rooms (PUT/DELETE)

- Oda pasifleştirilince veya silinince `room_closed` olayı yayınlanır.

/api/trtc/token ve /api/trtc/usersig

- Yanıta `trtcRoomId` (kanonik TRTC oda kimliği) ve `numericUid` eklendi.
- **Kritik kural:** Backend, TRTC oda kimliğini `voice_room_<chatRoomId>` biçiminde üretir. Hem web hem Flutter bu `trtcRoomId` değerini kullanmalı ki **aynı TRTC odasına** düşüp birbirini duysunlar. Flutter kendi başına oda kimliği üretmemeli.

4. Flutter'ın kullanacağı API listesi

Amaç	Çağrı
Oda listesi	GET /api/chat/rooms
Odaya tam durum (giriş anı)	GET /api/chat/rooms/{roomId}/state
Odaya katılma	POST /api/chat/rooms/{roomId}/presence (body: {nickname?, seatIndex?})
Heartbeat (canlı kalma)	POST /api/chat/rooms/{roomId}/presence (periyodik, ~20-30 sn)
Odadan ayrılma	DELETE /api/chat/rooms/{roomId}/presence? leave=1
Koltuk haritası	GET /api/chat/rooms/{roomId}/seats
Koltuk alma/değiştirme	PATCH /api/chat/rooms/{roomId}/seats (body: {seatIndex, targetUserId?})
Mikrofon aç	POST /api/chat/rooms/{roomId}/voice (body: {type:"join"})
Mikrofon kapat	POST /api/chat/rooms/{roomId}/voice (body: {type:"leave"})
Aktif ses kullanıcıları	GET /api/chat/rooms/{roomId}/voice
TRTC token	POST /api/trtc/token (body: {roomId, role?}) → trtcRoomId + numericUid kullan
Sahiplik devri	POST /api/chat/rooms/{roomId}/transfer-own- ership (body: {newOwnerId})
Gerçek zamanlı akış	GET /api/chat/rooms/{roomId}/stream (SSE)

Kimlik doğrulama: Flutter `Authorization: Bearer <JWT>` gönderir; web oturum çerezini kullanır. Tüm bu endpoint'ler her ikisini de kabul eder.

5. Gerçek zamanlı olay listesi (SSE)

SSE akışı (`/stream`) şu tipleri gönderir. Web bilinmeyen tipleri yok sayar; Flutter `type` alanına göre işler.

Mevcut tipler: `connected` , `messages` , `system` , `gift` , `pk` , `presence` , `typing` , DJ olayları.

presence (güncellendi) — Aktif kullanıcı listesi. Her kullanıcıda artık:

`id` , `name` , `image` , `nickname` , `seatIndex` , `micOn` , `chatRole` , `roleSymbol` , `roleLevel` , `isAdmin` , `lastSeen` .

room_event (yeni) — `event` alanına göre ayrıışan sesli oda olayları:

event	Ne zaman	Taşıdığı alanlar
<code>user_joined</code>	Kullanıcı odaya girdi	<code>userId</code> , <code>name</code> , <code>image</code>
<code>user_left</code>	Kullanıcı odadan çıktı	<code>userId</code> , <code>name</code>
<code>mic_changed</code>	Mikrofon açıldı/kapandı	<code>userId</code> , <code>micOn</code> , <code>name</code>
<code>seat_changed</code>	Koltuk değişti	<code>userId</code> , <code>seatIndex</code> , <code>previousSeatIndex</code>
<code>owner_changed</code>	Oda sahibi değişti	<code>newOwnerId</code> , <code>newOwnerName</code>
<code>room_closed</code>	Oda kapatıldı/silindi	<code>roomId</code>

Her olayda ayrıca `roomId` ve `ts` (zaman damgası) bulunur.

6. Senkronizasyon garantileri (gereksinim karşılığı)

- **Oda listesi:** Aynı `/api/chat/rooms` — aynı liste, sıralama, çevrimiçi sayısı, kapak.
- **Odaya giriş:** Tek `ChatPresence` kaydı; oda sahibi her iki platformda aynı gösterilir.
- **Odadan çıkış:** `lastSeen` anında sıfırlanır → listeden hemen düşer; oda değiştirince eski oda otomatik terk edilir (hayalet kullanıcı yok).
- **Koltuk sistemi:** Backend yönetir (`ChatPresence.seatIndex`); mikrofon/koltuk değişimi anlık yayınlanır.
- **Katılımcılar:** Aynı sıralama (rol seviyesi ↓ , sonra takma ad ↑); profiller backend'den; canlı güncelleme.
- **TRTC:** Oda kimliği (`voice_room_<id>`), kullanıcı sayısal UID'si ve token backend'de üretilir; web ve Flutter aynı `trtcRoomId` + `numericUid` bilgisini kullanır.

7. Test sonuçları

- **TypeScript tip kontrolü (`tsc --noEmit`):** Başarılı — hata yok (exit 0).
- **Üretim derlemesi (production build):** Başarılı — tüm sayfalar/route'lar derlendi (exit 0).
- **Dev sunucu:** Başarıyla ayağa kalktı; ana sayfa HTTP 200 döndü.
- **Şema:** Değişiklik yok → veritabanı migration'ı gerekmez, mevcut veri korunur.
- Derleme çıktısındaki `verify-device` uyarısı ve ana sayfadaki tekrar eden avatar görseli bu görevle ilgisiz, önceden var olan durumlardır.

Not: Derleme ve tip kontrolü doğrulandı. Canlı iki-cihaz (web + Flutter) uçtan uca ses testi yapılmadı; istenirse yayına alıp birlikte doğrulayabiliriz.